Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
A CHRISTIAN MINORITY RESIDENTIAL INSTITUTION
AICTE Approved & NAAC Accredited

DIVISION OF DATA SCIENCE AND CYBER SECURITY

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

A SKILL BASED EVALUATION REPORT

Malware Detection on EMBER2024 with Manual Tuning vs AutoML

SUBMITTED BY

ISAAC BENJAMIN DAVID(URK23AI1035)

COURSE CODE
25DC2005

COURSE NAME
AutoML

OCTOBER 2026

CERTIFICATES

TITLE

Malware Detection on EMBER2024 with Manual Tuning vs AutoML

Submitted by

ISAAC BENJAMIN DAVID(URK23AI1035)



DIVISION OF DATA SCIENCE AND CYBER SECURITY

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed-to-be-under Sec-3 of the UGC Act, 1956) Karunya Nagar, Coimbatore - 641 114. INDIA

OCTOBER 2026.

ABSTRACT

This project presents a comparative study of manual hyperparameter tuning and Automated Machine Learning (AutoML) for static malware detection. The task is to label Windows 64-bit Portable Executable (PE) files as benign or malicious from features extracted without running them. The project uses the EMBER2024 dataset (its Win64 part), in which every file is described by 2,568 numeric features in 12 families. It also asks a research question: how much does a random train/test split overstate real-world performance, and do AutoML tools protect against data leakage and temporal drift, or fall for them?

After removing duplicate files, a stratified sample of 160,000 training files and a random test set of 40,000 files were drawn from weeks 0-51, and a future test set of 40,000 files from the 12 later weeks, with a fixed random seed of 42. LightGBM with default settings and a logistic-regression model serve as baselines. LightGBM was then tuned with Grid Search, Random Search and Optuna, each with 36 configurations and 3-fold cross-validation. FLAML, AutoGluon and H2O AutoML were each given a 20-minute budget on the same training data.

The main metric is the share of malware caught when only 1% of benign files may be flagged (TPR at 1% FPR), with ROC-AUC, F1, accuracy and time, and 95% bootstrap confidence intervals. On the future test set, tuned LightGBM caught 97.6% of malware; AutoGluon was statistically tied with it (+0.05 points, 95% CI -0.15 to +0.24) in 20.2 minutes against 76.0 for the selected manual search, while FLAML and H2O AutoML were slightly but significantly behind tuned LightGBM. The random split overstated the tuned model's AUC by only 0.0010, but a planted leaky feature was used as the top input by 4 of 4 models and no AutoML tool removed it or mentioned it in its logs.

SHAP explains the tuned model's decisions. The tuned LightGBM was exported with a cost-based threshold and deployed through a FastAPI service and a Streamlit presentation page, with a model card. The study shows where AutoML matches careful manual work, and where human judgement is still needed: choosing honest test splits, spotting leaks, and setting thresholds for a realistic share of malware.

CHAPTER 1
INTRODUCTION

Malware detection is one of the main uses of machine learning in security. New malicious programs appear every day, and a detector that only matches known signatures misses them. Static detection reads the structure of a file (its headers, sections, imported functions, embedded text and digital signature) without running it, and a model learns which patterns separate malware from benign software. Building such a model involves data preparation, model selection, hyperparameter tuning, evaluation and deployment, which take expertise and time. Automated Machine Learning (AutoML) aims to automate model selection and tuning.

This project, titled **"Malware Detection on EMBER2024 with Manual Tuning vs AutoML,"** compares the two approaches on the EMBER2024 benchmark, a public dataset of Windows programs labelled as malware or benign. Each file is described by 2,568 numbers produced by the dataset's feature extractor (thrember). The prediction task is binary classification.

The primary objective is to evaluate whether AutoML frameworks (FLAML, AutoGluon and H2O AutoML) can match or beat a carefully tuned LightGBM model, the standard model for EMBER. LightGBM is tuned with Grid Search, Random Search and Optuna under an equal budget, and the AutoML tools receive the same training data and an equal time budget each.

A second objective is to test how honest the usual evaluation is. Every model is scored on two test sets: a random test set from the same weeks as the training data, and a future test set from later weeks, which is closer to real use. The project also plants leaky features on purpose to see whether any tool notices them, measures performance on slices of the data, explains the model with SHAP, and checks calibration and drift.

The main metric is the share of malware caught at a 1% false-positive rate, because a security team can only handle a limited number of false alarms. Cross-validation on the training data is used for every choice; the two test sets are used only for the final scores. The chosen model is served through a FastAPI service and a Streamlit presentation page.

Overall, the project aims to give a practical understanding of the strengths and limits of manual tuning and AutoML, and of the evaluation practices that decide whether a malware detector can be trusted.

CHAPTER 2
LITERATURE REVIEW

Machine learning has been widely applied to malware detection from static features. Traditional approaches require the developer to choose an algorithm, tune its hyperparameters and design an evaluation. These steps give control over the modelling process but take extensive experimentation. Automated Machine Learning reduces this effort by automating model selection and tuning.

Static Malware Detection and the EMBER Datasets

EMBER (Anderson and Roth, 2018) released features of over a million Windows PE files with a LightGBM baseline, and became the standard benchmark for static detection. EMBER2024 (Joyce et al., 2025) updates it with files from September 2023 to December 2024, weekly releases, several file types and a "challenge set" of malware that antivirus engines first missed. Its feature extractor, thrember, turns each file into a fixed-length vector of byte statistics, header fields, section, import and export information, strings and signature details.

Manual Machine Learning and Hyperparameter Optimization

Gradient-boosted decision trees are strong on tabular data. LightGBM (Ke et al., 2017) builds trees leaf-wise with histogram-based splits, which makes it fast on wide feature sets such as EMBER. Grid Search evaluates a fixed set of combinations, Random Search samples combinations at random and can reach similar results with fewer trials, and Optuna (Akiba et al., 2019) uses the Tree-structured Parzen Estimator to propose promising configurations from earlier trials. Scikit-learn (Pedregosa et al., 2011) provides the cross-validation, logistic regression and metric tools.

Automated Machine Learning Frameworks

FLAML (Wang et al., 2021) searches learners and hyperparameters while accounting for training cost. AutoGluon-Tabular (Erickson et al., 2020) trains many model types with presets and combines them by stacking and weighted ensembling. H2O AutoML (LeDell and Poirier, 2020) trains a sequence of algorithms and builds stacked ensembles of the best of each family. All three reduce manual work, but their results depend on the time budget, the hardware and how they validate internally.

Model Evaluation and Explainability

Reliable evaluation is essential when comparing approaches. For malware detection the true-positive rate at a low false-positive rate matters more than accuracy, and precision depends strongly on how common malware is. Paired bootstrap confidence intervals show whether a difference between two models is larger than the noise of the test set. SHAP (Lundberg and Lee, 2017), and its exact TreeSHAP form for tree models, explains how each feature contributed to a prediction; such explanations help understanding but do not prove a model correct. Model cards (Mitchell et al., 2019) document intended use, metrics and limits.

Temporal Drift and Data Leakage

Malware changes over time, so a random split, which mixes old and new files in both training and test data, can overstate real performance. TESSERACT (Pendlebury et al., 2019) showed that time-ignorant evaluation inflates malware-detection results, and recommended testing on files that are strictly later than the training data. Data leakage, where a feature carries information about the label that would not exist in real use, can produce misleadingly high scores; deliberately adding a leaky feature tests whether a pipeline, or an AutoML tool, notices it.

In summary, existing libraries and AutoML frameworks offer different ways to build a detector. Manual methods give more direct control, whereas AutoML can reduce experimentation time and explore many configurations. This project brings both together on the same malware-detection task, and evaluates them on both a random and a future test set.

CHAPTER 3

SYSTEM DESIGN

3.1 Architecture and high-level design

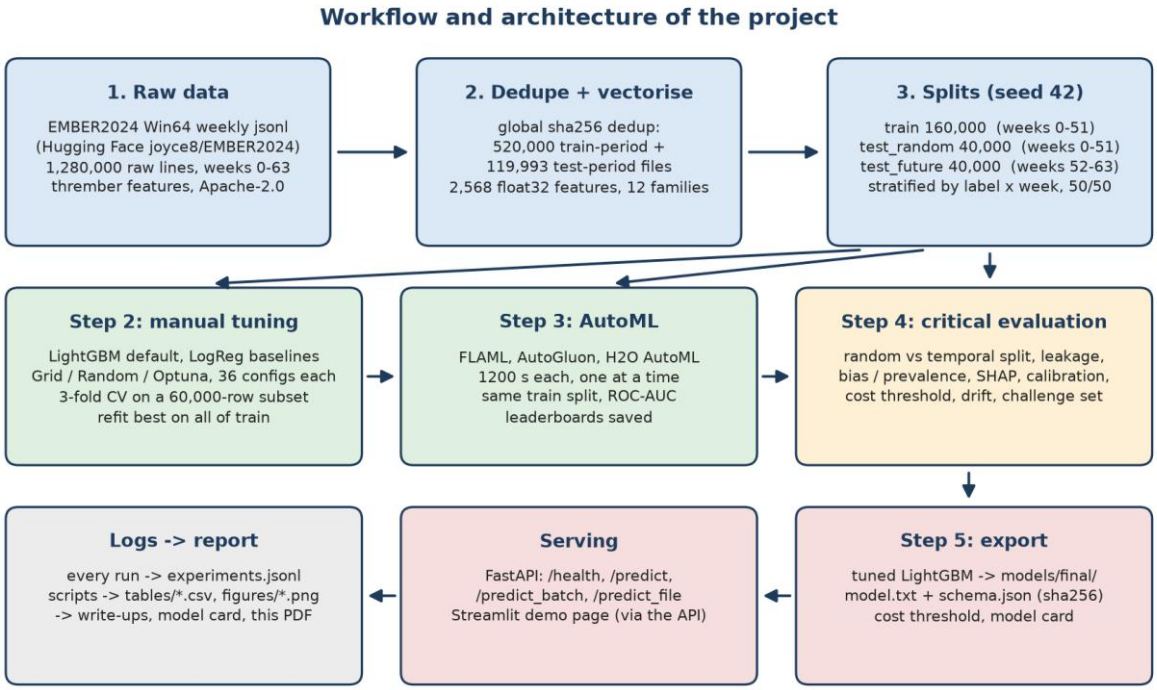


Fig 3.1.1 System diagram

The system has four layers.

1. Data layer. The EMBER2024 Win64 files (1,280,000 raw records in weekly files) are downloaded from Hugging Face with checksum checks. src/data.py removes duplicate files by sha256, extracts the feature vectors with thrember into a cache, and makes the fixed splits (train 160,000, test_random 40,000, test_future 40,000) with a random seed of 42.
2. Machine learning layer (src/). It has four stages:
    - Step 2 builds the LightGBM and logistic-regression baselines and tunes LightGBM with Grid Search, Random Search and Optuna (src/tune.py).
    - Step 3 runs FLAML, AutoGluon and H2O AutoML with the same time budget (src/automl_runs.py).
    - Step 4 does the critical evaluation: random vs future split, leakage experiments, bias slices, SHAP, calibration, a cost-based threshold and drift.

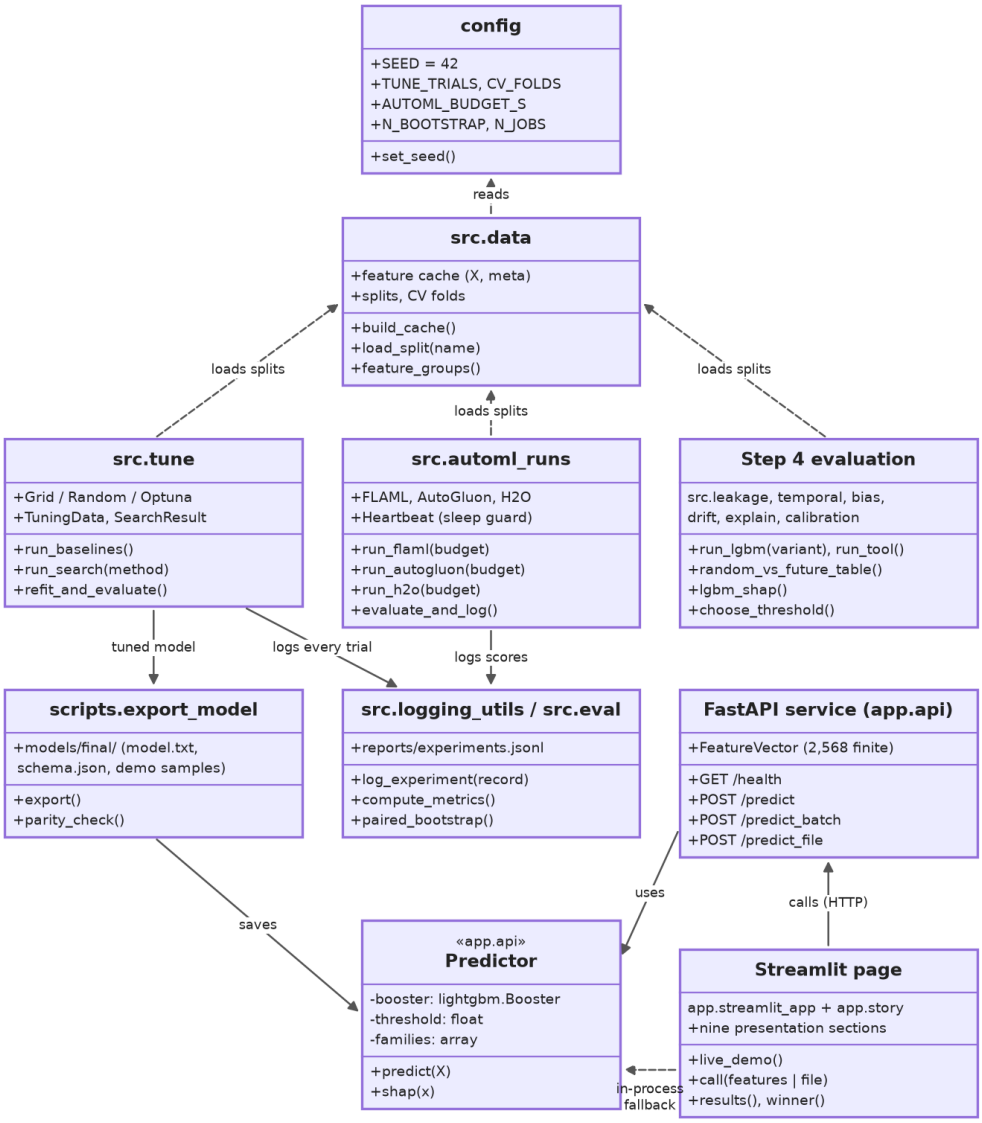
    - Step 5 exports the selected model with its schema and threshold (scripts/export_model.py).
3. Artifact layer. The deployed model is saved in models/final/ (model.txt, schema.json with its sha256 and threshold). Every run appends a record to reports/experiments.jsonl, and scripts turn it into tables, figures, the model card and this report.
4. Serving layer (app/). A FastAPI backend exposes the model as a web service, and a Streamlit page presents the project and calls the API for live predictions. Both use the same saved model.

Design decisions.

- Tuning and model selection use cross-validation on the training data only; both test sets stay unseen.
- The pipeline is modular: each step is a separate script that reads and writes files, so any step can be rerun alone (run_all.py runs them in order).
- One configuration file (config.py) holds the seed, sizes, budgets and paths.
- Every number in the reports is generated from the experiment log and saved predictions, never typed by hand.

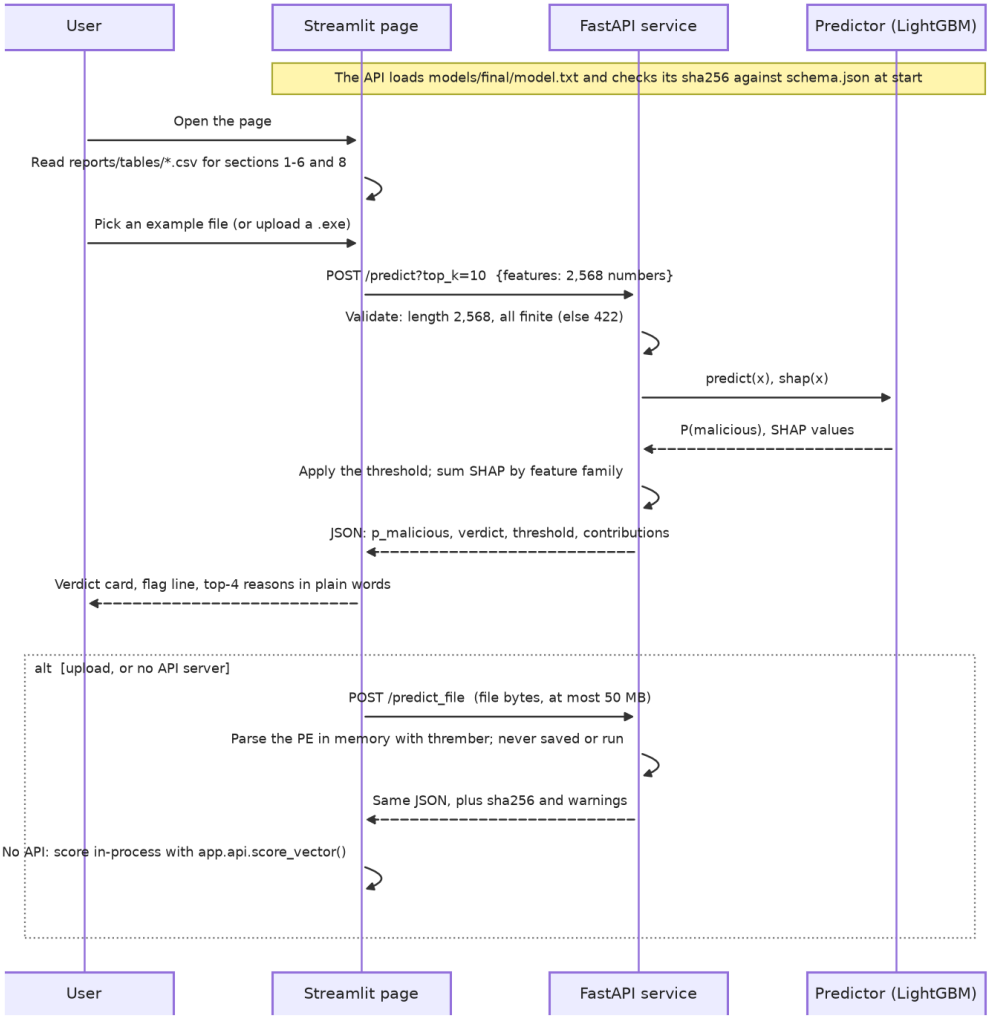
3.2 UML diagrams

Class diagram



The project is mostly written as Python modules and functions, so the diagram shows each module as a component class. The configuration and data modules feed the tuning, AutoML and evaluation steps; every step logs to the experiment log. The export step saves the tuned LightGBM, which the Predictor class of the API loads and serves. The Streamlit page calls the API, or uses the same Predictor in-process when no API server is running.

Sequence diagram



A user opens the page, which reads the saved result tables for the presentation sections, and picks an example file. The page sends its 2,568 features to POST /predict. The API checks that the vector has the right length and only finite values, scores it with the LightGBM model, computes TreeSHAP values, applies the threshold and sums the SHAP values by feature family. The page shows the verdict, the flag line (the score above which a file is flagged) and the four families that pushed the decision most. As an alternative path, an uploaded program goes to POST /predict_file, where it is parsed in memory and never saved or run.

3.3 User interface design (Streamlit)

The interface is a single web page titled "Can a model spot malware without running it?" that reads top to bottom as the whole presentation, in ten numbered sections, with no tabs or side panels.

- **Story sections.** The question, the data (with a timeline of the training and future weeks), the contenders, how they were compared, the results (a chart of every model with 95% confidence intervals), a full scorecard (AUC, accuracy, precision, recall, F1

and false-alarm rate for every model on both tests, the deployed model's confusion matrix, and the cross-validation scores), the winner and why, what was learned, and the limits. Every number is read from reports/tables/.

- **Try it live.** Five buttons pick a caught malware file, a safe program, a missed malware file, a false alarm, or an upload of a Windows program. The page shows a red or green verdict card with the malware score, the flag line and the true answer, then the feature families that pushed the decision, in plain words.
- **Technical detail.** A box at the end of the demo shows the raw probability, the threshold, the file's sha256 and the top-10 SHAP columns, for questions.

The FastAPI backend has its own interface at /docs (Swagger UI), where requests can be tried and the JSON response seen directly. present.py starts the API and the page together.

CHAPTER 4

IMPLEMENTATION

The implementation of the project, *Malware Detection on EMBER2024 with Manual Tuning vs AutoML*, focuses on building and comparing manually tuned and AutoML models for detecting malicious Windows programs. It is written in Python 3.12, and every step is a script that can be rerun on its own.

1. Dataset Loading and Preprocessing

The EMBER2024 Win64 files are downloaded from Hugging Face and checked. Each weekly file repeats every sample, so 640,000 of the 1,280,000 raw lines were duplicates within a file, and 7 more files appeared in two different weeks; all were removed by sha256, keeping the first occurrence and leaving 520,000 files in the training period and 119,993 in the test period. thrember turns each file into 2,568 float32 features, saved in a memory-mapped cache with the label, week and metadata. The splits are stratified by label and week: train 160,000 and test_random 40,000 from weeks 0-51, and test_future 40,000 from weeks 52-63, all with seed 42. The leaky detection_ratio and first_submission_date fields are kept in the metadata only, never in the features.

2. Manual Machine Learning Implementation

LightGBM with default settings and a logistic regression on standardized features are the baselines. LightGBM is tuned with Grid Search, Random Search and Optuna (TPE sampler), each evaluating 36 configurations with the same 3-fold cross-validation folds on a 60,000-row subset of the training data, with early stopping inside each fold. The configuration with the best mean cross-validated ROC-AUC is refit on all training rows with a fixed number of rounds.

3. AutoML Implementation

FLAML, AutoGluon (medium-quality preset) and H2O AutoML each receive the same 160,000 training rows and a 20-minute budget, with ROC-AUC as the metric. Seed 42 is passed to each tool's main seed argument, although some internal models keep the library's own fixed seed. Each tool chooses its own internal validation inside the training data. The tools run one at a time on the same machine, and a heartbeat guard records whether the machine slept, which would invalidate a timed run. Each tool's leaderboard, preprocessing notes and feature importance are saved.

4. Model Evaluation and Explainability

Every model is scored on both test sets with ROC-AUC, F1, accuracy, false-positive rate and TPR at 1% FPR, with 95% confidence intervals from 1,000 paired bootstrap resamples. The critical evaluation compares stratified and forward (week-blocked) cross-validation, reruns the tuned model and every AutoML tool (10-minute budget) with and without each leaky feature, measures recall and false alarms per slice (week, file size, packer, signed or unsigned), tries class weights, explains the model with TreeSHAP, checks calibration, chooses a cost-based threshold and tracks drift over the future weeks.

5. Application Development

The tuned LightGBM (609 trees) is exported to models/final/ with schema.json, which records its sha256, the feature layout and the threshold (0.0069, chosen so that a missed malware costs 10 times a false alarm, on training out-of-fold scores only). This threshold is deliberately low: on the future test set it catches 99.5% of malware but flags 7.4% of benign files. A FastAPI service serves /health, /predict, /predict_batch and /predict_file, and a Streamlit page presents the project and the live demo.

CHAPTER 5

TESTING AND VALIDATION

Testing and validation are essential parts of the project, *Malware Detection on EMBER2024 with Manual Tuning vs AutoML*. They verify that the data and code are correct, that no test information leaks into model choices, and that the application returns the same predictions as the offline model.

1. Data Validation

scripts/verify_data.py writes reports/data_check.json. It checks that every downloaded file has the expected number of lines, that labels are only 0 or 1, that the 12 feature groups add up to 2,568 columns, that features and metadata stay aligned, and that the splits do not overlap (0 shared sha256 across them). Every future-test file is dated after every training file: training covers 2023-09-24 to 2024-09-21 and the future test 2024-09-22 to 2024-12-14.

2. Model Validation

The training set is used for model development and hyperparameter tuning; both test sets are reserved for final evaluation. All tuning methods share the same saved cross-validation folds. A week-blocked forward cross-validation was compared with the stratified one: stratified CV gave AUC 0.9988 and forward CV 0.9975, and their TPR at 1% FPR missed the future-test value by 0.74 and 0.98 points.

3. Performance Testing

All eight models (two baselines, three tuned LightGBM searches and three AutoML tools) are evaluated with the same metric code (src/eval.py). The primary metric is TPR at 1% FPR, supported by ROC-AUC, F1, accuracy and false-positive rate. Paired bootstrap confidence intervals decide whether two models really differ.

4. Overfitting and Data Leakage Testing

The gap between the random and future test sets shows how much a random split overstates performance: 0.0010 AUC for tuned LightGBM, 95% CI [0.0007, 0.0014]. In the leakage experiment, the detection_ratio field (the share of antivirus engines that flag the file) is added as a feature, then removed; a second, subtler variant adds the first-submission date. The tuned model is also retrained without the time-proxy columns (compile and signing times) to check that they are not a hidden leak.

5. Application and Functional Testing

Pytest covers split disjointness and temporal order, the tuners and metrics on synthetic data, the AutoML and leakage table code, and the application. The API tests check that a wrong-length or non-finite vector is rejected with HTTP 422, that an uploaded file is never written to disk, and that the API returns exactly the offline model's scores. The page tests render every section and compare the numbers shown with the result tables.

6. Validation Summary

The final validation uses the two held-out test sets to assess every model on unseen data. A reproducibility check rebuilt a fresh virtual environment, ran the test suite and the quick pipeline end to end (reports/repro_check.md). The conclusions are based on the measured results rather than assumed values.

CHAPTER 6

RESULTS AND DISCUSSION

The project, *Malware Detection on EMBER2024 with Manual Tuning vs AutoML*, compares manual tuning with AutoML for detecting malicious Windows programs. The evaluation considers predictive performance on both test sets, tuning methods, time, reliability and interpretability.

1. Comparison of Model Performance

Table 6.1 lists every model, ordered by the main metric on the future test set.

Model	AUC (random test)	AUC (future test)	TPR at 1% FPR (future)	F1 (future)	Time (min)
AutoGluon	0.9991	0.9980	97.6%	0.9834	20.2
LightGBM, Optuna (tuned)	0.9991	0.9981	97.6%	0.9838	76.0
LightGBM, random search	0.9990	0.9980	97.5%	0.9827	90.8
LightGBM, grid search	0.9991	0.9980	97.4%	0.9827	116.6
FLAML	0.9986	0.9976	96.5%	0.9786	25.7
H2O AutoML	0.9984	0.9970	96.1%	0.9771	24.8
LightGBM, default settings	0.9978	0.9967	95.1%	0.9744	0.4
Logistic regression	0.9929	0.9297	81.7%	0.9172	0.3

Table 6.1 Results on both test sets (reports/tables/random_vs_future.csv, automl_vs_manual.csv)

The best models are tuned LightGBM (97.6% of future malware caught) and AutoGluon. Their difference, +0.05 points (95% CI -0.15 to +0.24), includes zero, so they are tied. FLAML (-1.12 points) and H2O AutoML (-1.47 points) are slightly but significantly behind tuned LightGBM. Logistic regression reaches only 0.9297 AUC on the future test set.

2. Effect of Hyperparameter Optimization

Tuning helped: on the future test set the share of malware caught rose from 95.1% with default settings to 97.6% for the Optuna-tuned model. The search strategy mattered much less: Grid, Random and Optuna ended within 0.19 points of each other. Optuna had the best cross-validated AUC and was therefore selected, without looking at any test score.

3. Performance of AutoML Approaches

AutoGluon matched the tuned model in 20.2 minutes, against 76.0 minutes for the selected manual search and 283.5 minutes for all three manual searches. All three tools ended up with boosted decision trees, the same kind of model as the manual approach (FLAML: XGBoost, a

boosted-tree model; AutoGluon: an ensemble that put all its weight on one LightGBM model; H2O AutoML: an ensemble that in effect used one boosted-tree model (GBM)). The time comparison is not like for like: the manual searches had no time cap and searched one model family on a 60,000-row subset, while the AutoML budget is a soft limit (FLAML took 25.7 and H2O AutoML 24.8 minutes in total).

4. Overfitting and Data Leakage Analysis

By AUC, the random split overstates performance only slightly for tree models (gaps of 0.0010 to 0.0014), but much more for logistic regression (0.0632). The larger effect is the base rate: the tests are half malware, so 93.1% of the deployed model's alarms are real, but if only 1% of files were malware about 11.9% would be, about 7.4 false alarms per real one. In the leakage experiment, 4 of 4 models made detection_ratio their most important input, and their future-test AUCs rose to between 0.9999 and 1.0000. All 3 AutoML tools kept the column and none mentioned it in their logs. The first-submission date moved the future-test AUC by at most 0.0009, and removing the time-proxy columns changed it by -0.0002.

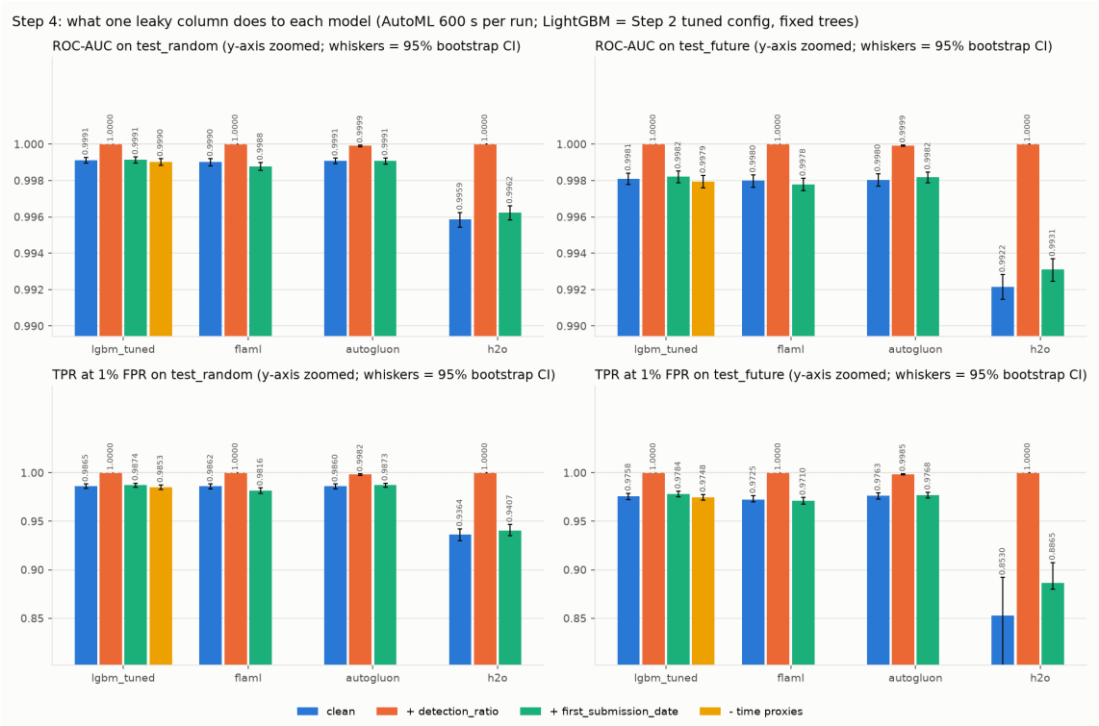


Fig 6.1 Scores with and without the leaky features (reports/figures/leakage_jump.png)

5. Interpretability and Fairness

SHAP shows that the tuned model relies most on the strings (17%), imports (15%), histogram (13%) feature families. Recall differs between slices: at a 1% false-positive threshold the model caught 83.7% of signed malware (970 files) against 98.6% of unsigned malware. On the 814 challenge-set files, malware that antivirus engines first missed, the deployed model caught 79.9%, against 99.5% of ordinary future malware. Balanced class weights changed the

main metric by -0.02 points (95% CI [-0.20, +0.14]). The tuned model's probabilities are well calibrated: Brier score 0.0132, against 0.0700 for logistic regression.

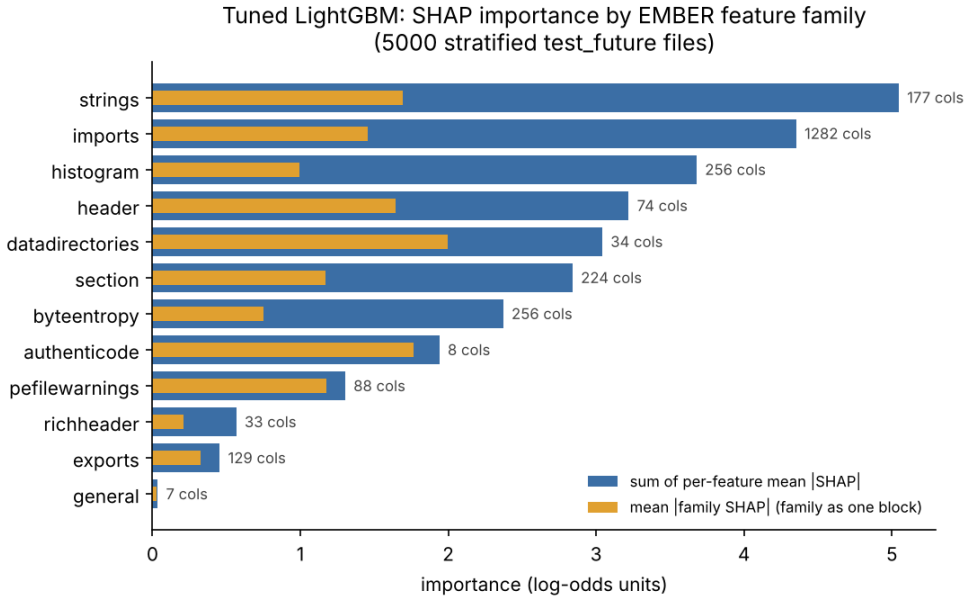


Fig 6.2 SHAP importance by feature family (reports/figures/shap_family_bar.png)

Discussion

The comparison shows that a good AutoML tool can match careful manual tuning on accuracy in less wall-clock time, while the manual model is simpler to deploy: one file, chosen by cross-validation, that needs no AutoML library and is explained with TreeSHAP. The evaluation choices mattered more than the choice between manual tuning and AutoML: testing on later weeks, reporting precision at a realistic base rate, and checking for leaks changed the picture far more than the differences between the top models.

CONCLUSION

The project, *Malware Detection on EMBER2024 with Manual Tuning vs AutoML*, evaluated manual hyperparameter tuning and Automated Machine Learning for detecting malicious Windows programs from static features. Using the EMBER2024 Win64 data, it provides a structured, reproducible approach to data preparation, tuning, AutoML, critical evaluation and deployment.

The implementation includes LightGBM and logistic-regression baselines, LightGBM tuned with Grid Search, Random Search and Optuna, and the AutoML frameworks FLAML, AutoGluon and H2O AutoML under an equal time budget. Tuned LightGBM caught 97.6% of future malware at a 1% false-positive rate, and AutoGluon was statistically tied with it (+0.05 points) in 20.2 minutes.

The project also highlights reliable evaluation practices: a future test set, paired bootstrap confidence intervals, leakage experiments, slice metrics and calibration. A planted leak was used as the top input by 4 of 4 models and no AutoML tool removed it or mentioned it in its logs; and at a realistic 1% share of malware most alarms would be false. SHAP explains the model's decisions, and the FastAPI service and Streamlit page show how the trained model can be used in an application.

Overall, the project shows that AutoML can match careful manual work on accuracy and saves time, but does not replace human judgement about how to test a model honestly, how to spot leaks, and where to set the decision threshold.

In the future, the project can be extended by repeating the AutoML runs with several seeds to measure their run-to-run variation, testing against adversarially modified malware, evaluating other file types in EMBER2024, retraining on a sliding window to follow drift, and deploying the application on a cloud platform with authentication.

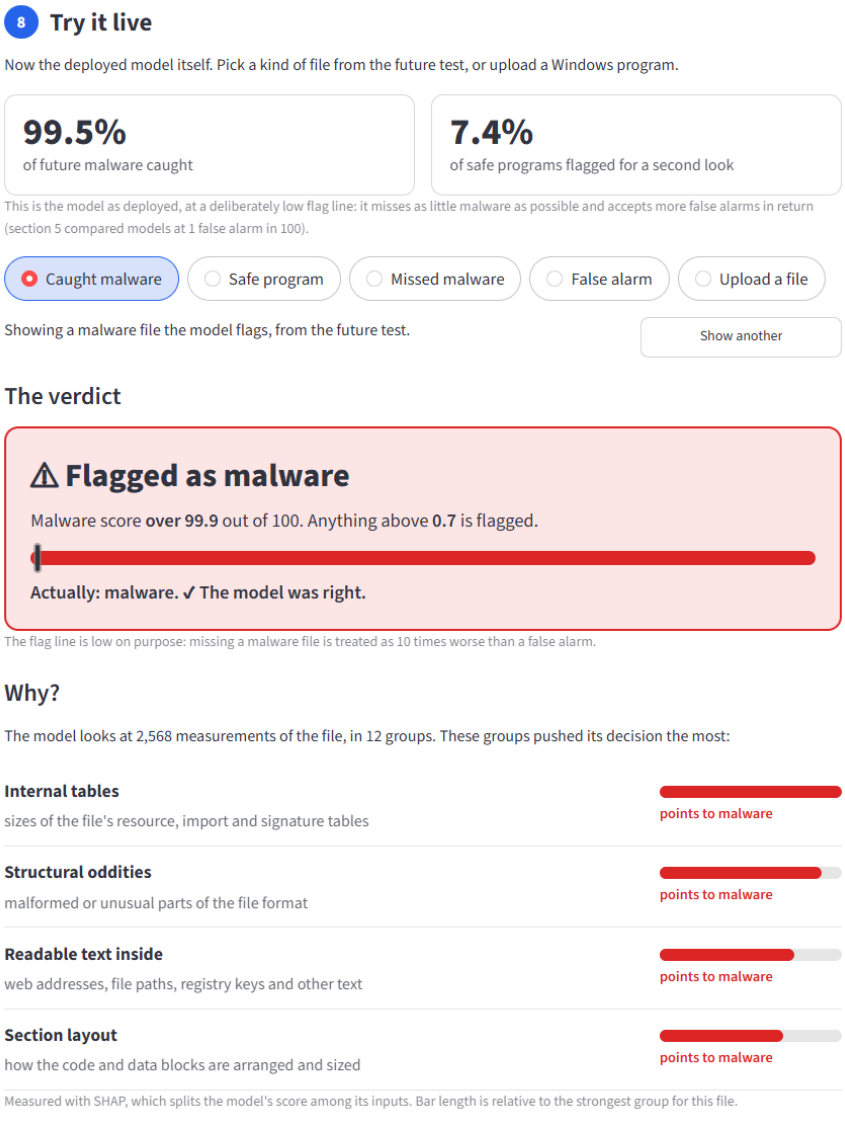
REFERENCES

1. Joyce, R. J., Miller, G., Roth, P., Zak, R., Zaresky-Williams, E., Anderson, H., Raff, E., and Holt, J. (2025). *EMBER2024 - A Benchmark Dataset for Holistic Evaluation of Malware Classifiers.* Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining. Available at: https://arxiv.org/abs/2506.05074
2. Anderson, H. S., and Roth, P. (2018). *EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models.* arXiv preprint arXiv:1804.04637. Available at: https://arxiv.org/abs/1804.04637
3. Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems, 30.* Available at: https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html
4. Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining.* Available at: https://doi.org/10.1145/3292500.3330701
5. Wang, C., Wu, Q., Weimer, M., and Zhu, E. (2021). FLAML: A Fast and Lightweight AutoML Library. *Proceedings of Machine Learning and Systems, 3.* Available at: https://arxiv.org/abs/1911.04706
6. Erickson, N., Mueller, J., Shirkov, A., et al. (2020). AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data. *arXiv preprint arXiv:2003.06505.* Available at: https://arxiv.org/abs/2003.06505
7. LeDell, E., and Poirier, S. (2020). H2O AutoML: Scalable Automatic Machine Learning. *7th ICML Workshop on Automated Machine Learning.* Documentation available at: https://docs.h2o.ai/h2o/latest-stable/h2o-docs/automl.html
8. Lundberg, S. M., and Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems, 30.* Available at: https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

9. Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., and Cavallaro, L. (2019). TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time. *28th USENIX Security Symposium.* Available at: https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury
10. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830. Available at: https://jmlr.org/papers/v12/pedregosa11a.html
11. Mitchell, M., Wu, S., Zaldivar, A., et al. (2019). Model Cards for Model Reporting. *Proceedings of the Conference on Fairness, Accountability, and Transparency.* Available at: https://doi.org/10.1145/3287560.3287596
12. FastAPI. *FastAPI Documentation.* Available at: https://fastapi.tiangolo.com/
13. Streamlit. *Streamlit Documentation.* Available at: https://docs.streamlit.io/

APPENDICES

SCREENSHOTS



8 Try it live

Now the deployed model itself. Pick a kind of file from the future test, or upload a Windows program.

99.5%
of future malware caught

7.4%
of safe programs flagged for a second look

This is the model as deployed, at a deliberately low flag line: it misses as little malware as possible and accepts more false alarms in return (section 5 compared models at 1 false alarm in 100).

○ Caught malware ○ Safe program ○ Missed malware ○ False alarm ◉ Upload a file

A Windows program (.exe or .dll). It is read in memory, never run or saved.

notepad.exe 352.0KB +

The verdict

✓ Looks safe

Malware score **0.5** out of 100. Anything above **0.7** is flagged.

The true answer is not known for an uploaded file.

The flag line is low on purpose: missing a malware file is treated as 10 times worse than a false alarm.

Why?

The model looks at 2,568 measurements of the file, in 12 groups. These groups pushed its decision the most:

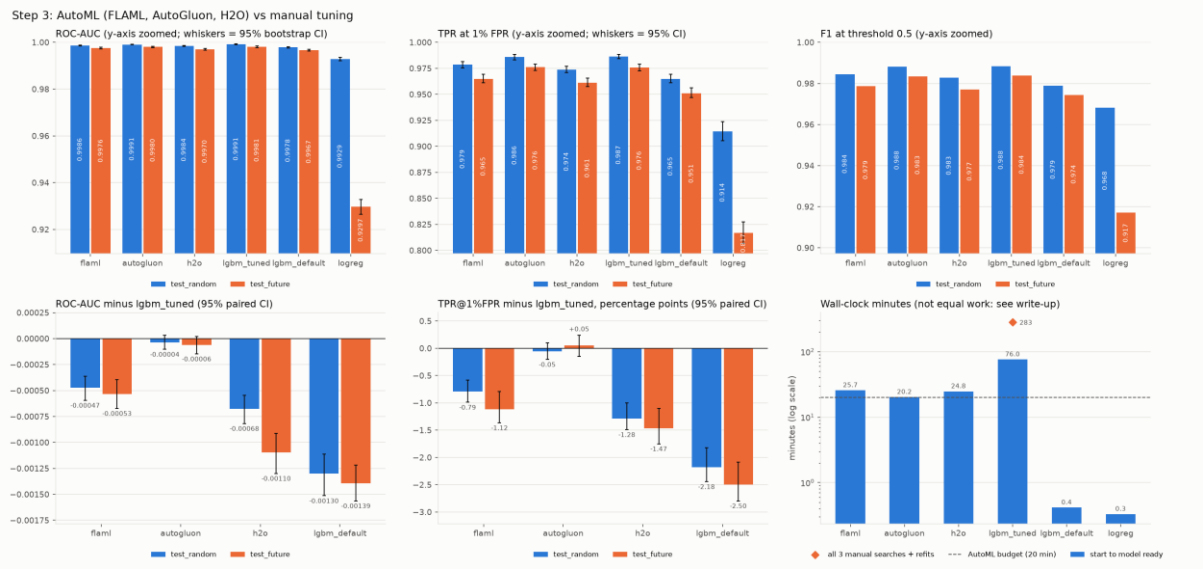
Windows functions it uses
for example networking, registry or process access
points to safe

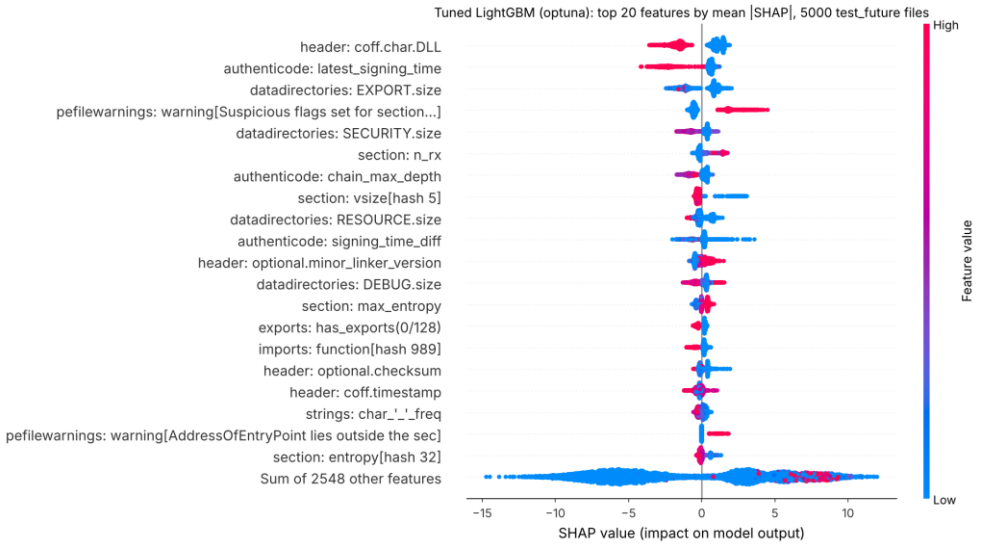
Section layout
how the code and data blocks are arranged and sized
points to safe

Internal tables
sizes of the file's resource, import and signature tables
points to safe

Structural oddities
malformed or unusual parts of the file format
points to safe

Measured with SHAP, which splits the model's score among its inputs. Bar length is relative to the strongest group for this file.




Tuned LightGBM (optuna): top 20 features by mean |SHAP|, 5000 test_future files
header: coff.char.DLL
authenticode: latest_signing_time
datadirectories: EXPORT.size
pefilewarnings: warning[Suspicious flags set for section...]
datadirectories: SECURITY.size
section: n_rx
authenticode: chain_max_depth
section: vsize[hash 5]
datadirectories: RESOURCE.size
authenticode: signing_time_diff
header: optional.minor_linker_version
datadirectories: DEBUG.size
section: max_entropy
exports: has_exports(0/128)
imports: function[hash 989]
header: optional.checksum
header: coff.timestamp
strings: char_'_'_freq
pefilewarnings: warning[AddressOfEntryPoint lies outside the sec]
section: entropy[hash 32]
Sum of 2548 other features
−15
−10
−5
0
5
10
SHAP value (impact on model output)
High
Feature value
Low

EVALUATION SHEET

Reg.No :URK23AI1035

Name:ISAAC BENJAMIN DAVID

Course code: 25DC2005

Course Name: AUTOML

S.No	Rubrics	Maximum Marks	Marks Obtained
1	Online Certification	10	
2	Baseline model and hyperparameter tuning	5	
3	AutoML implementation and tool comparison	10	
4	Critical evaluation	5	
5	Deployment	5	
6	Documentation and presentation	5	
	Total	**40**	

Signature of the Faculty-in-charge